42 ft_printf: implementation and memory guide

A complete basic implementation of the mandatory conversions, with a line-by-line design explanation.

1. What the project does

ft_printf reproduces the core behavior of printf: it scans a format string, detects conversion markers beginning with %, retrieves the matching argument from a variadic argument list, writes the result to standard output, and returns the number of characters written. This version implements the mandatory 42 conversions: c, s, p, d, i, u, x, X, and %.

2. Files included

File	Purpose
ft_printf.h	Public prototype and standard headers
ft_printf.c	Format parser, conversions, output, and counting
Makefile	Builds libftprintf.a with 42-style flags

3. Building and testing

```
make
cc -Wall -Wextra -Werror -I. test.c libftprintf.a -o test
./test
make clean
make fclean
```

make creates the static library libftprintf.a. A user program includes ft_printf.h and links against that archive. clean removes object files; fclean also removes the library; re rebuilds everything.

4. Format parsing

ft_printf walks through format one byte at a time. Ordinary characters are written immediately. On %, the pointer advances to the conversion letter and print_conversion selects the correct va_arg type. The exact type passed to va_arg matters: reading an unsigned int as an int, for example, is undefined behavior.

A trailing % is rejected with -1. Unknown conversions are also rejected with -1. After every successful write, the character count is accumulated. va_end is called on every return path after va_start.

5. Conversion behavior

Conversion	Argument	Implementation
%c	int	write one character
%s	char *	write until '\0'; NULL becomes (null)
%d / %i	int	signed decimal, including negatives
%u	unsigned int	unsigned decimal
%x / %X	unsigned int	lowercase/uppercase hexadecimal
%p	void *	0x followed by hexadecimal address
%%	none	write a literal percent sign

6. Number conversion and recursion

putn converts an unsigned number to a chosen base. It recursively prints n / base first, then prints the final digit n % base. This reverses the natural least-significant-digit order without allocating a temporary string or array. Base 10 uses the digits string 0123456789; hexadecimal uses the appropriate lowercase or uppercase alphabet.

print_signed widens int to long long before negating it. This avoids the classic INT_MIN problem: negating INT_MIN as an int is not representable, but its long long counterpart is. The sign is written separately and the magnitude is passed to putn.

7. Memory allocation in detail

This implementation performs no heap allocation: there is no malloc, calloc, realloc, or free. Every helper uses automatic (stack) variables such as integers, a character, and a va_list. The digit alphabets are string literals stored by the program; they are not copied or freed. Recursion uses stack frames temporarily, one frame per output digit, and each frame disappears automatically when it returns.

The absence of heap allocation eliminates leaks and allocation-failure paths. It also means the function does not return a pointer to internal storage: output is written directly to file descriptor 1. The trade-off is that recursion consumes stack space proportional to the number of digits. For normal int and unsigned int values this is at most a few dozen frames, which is safe.

va_list is not heap memory owned by this code. va_start initializes access to the arguments supplied by the caller; va_arg consumes one argument with the requested type; va_end releases any implementation-specific state. The caller still owns pointer arguments such as the string passed to %s. ft_printf only reads them and never frees them.

For %p, the pointer is converted to an unsigned integer-sized value for printing. The pointer itself is not dereferenced, copied, allocated, or freed. A NULL string is handled safely by substituting the literal (null); this substitution still does not allocate memory.

8. Error handling and ownership

write returns the number of bytes written or -1 on error. The helpers propagate write failures. ft_printf returns -1 for a NULL format, malformed format, unsupported conversion, or output failure. The function owns only its va_list state; the caller owns every argument and all pointed-to storage.

9. Complete source code

ft_printf.h

```
#ifndef FT_PRINTF_H
# define FT_PRINTF_H

# include <stdarg.h>
# include <stddef.h>

int■ft_printf(const char *format, ...);
int■ft_putchar_fd(char c, int fd);
int■ft_putstr_fd(const char *s, int fd);

#endif
```

ft_printf.c

```
#include "ft_printf.h"
#include <unistd.h>

static int putn(unsigned long long n, int base, const char *digits)
{
■char c;
■int written;
■if (n >= (unsigned long long)base)
■{
■■written = putn(n / base, base, digits);
■■if (written < 0) return (-1);
■}
■else
■■written = 0;
■c = digits[n % base];
■if (write(1, &c, 1) != 1) return (-1);
■return (written + 1);
}

int ft_putchar_fd(char c, int fd)
{
■return (write(fd, &c, 1) == 1 ? 1 : -1);
}

int ft_putstr_fd(const char *s, int fd)
{
■int n;
■if (!s) s = "(null)";
■n = 0;
■while (s[n])
■{
■■if (write(fd, &s[n], 1) != 1) return (-1);
■■n++;
■}
■return (n);
}
```

```
static int print_signed(int value)
{
	long long n;
	int count;
	n = value;
	count = 0;
	if (n < 0)
	{
		if (ft_putchar_fd('-', 1) < 0) return (-1);
		count++;
		n = -n;
	}
	if (n == 0) return (count + ft_putchar_fd('0', 1));
	count += putn((unsigned long long)n, 10, "0123456789");
	return (count < 0 ? -1 : count);
}

static int print_conversion(char spec, va_list *ap)
{
	if (spec == 'c') return (ft_putchar_fd(va_arg(*ap, int), 1));
	if (spec == 's') return (ft_putstr_fd(va_arg(*ap, char *), 1));
	if (spec == 'd' || spec == 'i') return (print_signed(va_arg(*ap, int)));
	if (spec == 'u') return (putn(va_arg(*ap, unsigned int), 10, "0123456789"));
	if (spec == 'x') return (putn(va_arg(*ap, unsigned int), 16, "0123456789abcdef"));
	if (spec == 'X') return (putn(va_arg(*ap, unsigned int), 16, "0123456789ABCDEF"));
	if (spec == 'p')
	{
		if (ft_putstr_fd("0x", 1) < 0) return (-1);
		return (2 + putn((unsigned long long)va_arg(*ap, void *), 16,
			"0123456789abcdef"));
	}
	if (spec == '%') return (ft_putchar_fd('%', 1));
	return (-2);
}

int ft_printf(const char *format, ...)
{
	va_list ap;
	int total;
	int result;
	if (!format) return (-1);
	va_start(ap, format);
	total = 0;
	while (*format)
	{
		if (*format != '%') result = ft_putchar_fd(*format, 1);
		else
		{
			format++;
			if (!*format) { va_end(ap); return (-1); }
			result = print_conversion(*format, &ap);
			if (result == -2) { va_end(ap); return (-1); }
		}
		if (result < 0) { va_end(ap); return (-1); }
		total += result;
		format++;
	}
	va_end(ap);
	return (total);
}
```

Makefile

```
NAME = libftprintf.a
CC = cc
CFLAGS = -Wall -Wextra -Werror
AR = ar rcs
RM = rm -f

SRC = ft_printf.c
OBJ = $(SRC:.c=.o)

all: $(NAME)

$(NAME): $(OBJ)
	$(AR) $(NAME) $(OBJ)

%.o: %.c ft_printf.h
	$(CC) $(CFLAGS) -c $< -o $@

clean:
	$(RM) $(OBJ)

fclean: clean
	$(RM) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

10. Scope and next steps

This is a mandatory-conversion implementation. It intentionally does not implement optional flags, field width, precision, length modifiers, or bonus conversions. Those features require a parsing state machine and additional formatting logic. Before submitting to 42, compare the exact behavior and allowed functions against your campus subject and run the official tester.